

AI 社区：已经做成了什么，接下来怎么把它盘活

六个板块的功能与技术盘点 · 三条发展路线的利弊与选择 · 十二项新功能的详细方案

① 已有能力

六个板块 + 后台架构，**每个板块一页**：功能亮点讲给业务听，技术亮点讲清楚难在哪、怎么解决的。包含刚上线的**投票评选与短视频双语字幕**。

第 3 — 13 页

② 发展方向

怎么盘活社区，有三条路：**真人社区** / 专家专栏 / AI 资源挂钩。三条摊开比利弊，给出选择与理由，以及主推那条怎么一步步落地。

第 14 — 17 页

③ 新功能与投入

十二项新功能，先一页总览再逐项展开（功能 + 技术）；最后是**三批交付节奏、怎么度量、需要哪些支持**。

第 18 — 32 页

今天想拿到的三个结论

其余都可以会后再定

- **确认走"真人社区"这条主线** —— 先让人和关系立起来，专家专栏做栏目、AI 额度做放大器，不三线并行。
- **第一批六项功能可以立即开工**（关注流、名片与红人、话题、一键发日常、新人引导、一页速递），零外部依赖。
- **两件需要拍板的事**：AI 额度池的授权，以及内网加载一个向量模型的算力排期。

平台已经建成并在内网运行。**这次汇报的重点不是"还能加什么功能"，而是"怎么让它有人用"。**

汇报提纲：三章，每章的结论先写在这里

中间要跳看这张表就行

章	内容	核心结论	页
一	已有能力 功能 + 技术	六个板块全部上线，一套账号 / 通知 / 评论 / 权限打通。技术上真正难的是四类事： 并发下的计数与配额 （投票、点赞、报名）、 权限的单点收敛与服务端裁剪 、 内网无外部依赖下的自建能力 （分发、字幕、出网分流）、 推理模型的工程容错 。	3 — 13
二	发展方向 三条路线	问题不是缺功能，是缺"发的动力"和"看的理由"。三条路线里 A 真人社区做主线 （人和关系是外部社区复制不了的资产），B 专家专栏降级成栏目，C AI 额度作为放大器 —— 因为 B 和 C 都能被 A 吸收 ，反过来不行。	14 — 17
三	新功能 十二项 + 投入	按"让人可见 → 让发容易 → 让留有理由"三步排。第一批六项零外部依赖、可立即开工；第二批依赖一个向量模型的算力排期；第三批是 MCP 出口与挑战赛这类加分项。	18 — 32

一句话总结全场：**东西都在，人不在** —— 所以这一轮不加板块，只做让人愿意来、愿意发、愿意留的机制。

【已有能力】一个社区，六个板块 —— 全部已经在内网跑起来

下表只列已上线能力，不含规划项；后面每个板块单独一页展开

板块	解决什么问题	核心能力	页
技能市场 含合集包	同一段提示词、同一个 Skill，几个人各写一遍	上传 / 评分 / 版本 / 订阅更新 · 公开-受限-私有三档 + 逐人审批 · 一行命令装进项目 · 不装先试 · 装与不装同题实测对比 · Remix 二创并署名	5
知识库	好资料散落各处；同一份长文档几个人各读一遍	链接 / PDF / EPUB / PPT / Word 入库 · AI 逐章导读 · 问答带原文引用可跳转 · 划线批注与共享笔记 · 选中即译 · 读到哪儿记得住	6
讨论区	群聊里刷过去就找不回，半年后无从检索	动态流 + 论坛话题两种形态 · 图片 / 视频 / 文档附件 · 六种表情反应 · 两级评论与深链定位 · 多主题归档	7
投票活动 最新上线	评选靠群里接龙、Excel 统计，票数说不清	作品征集 + 审核 · 每人限票与撤票改投 · 匿名评选 · 随机排序防前排优势 · 实名一人一票 · 实时榜单与明细导出	8
极客视频 含随刷短视频	技术分享看完就沉；长视频没人有耐心看完	抖音式竖屏随刷 · 人人可发短视频 · 自动生成中英双语字幕 · 长视频 AI 摘要与就视频问答 · 点赞收藏评论全站打通	10
社区活动	讲座只发一个邮件列表，隔壁实验室不知道	全员可见日历 · 按类型 / 主题 / 城市 / 形式筛 · 按浏览者时区自动换算 · 一键报名 + 开场前自动提醒 · 加进 Google / Outlook 日历	11

支撑层（第 12—13 页展开）：全站通知铃与邮件（按类型可开关）· 8K 跨 11 类内容搜索 · 需求反馈板 · 公告 · 个人主页与私人工作台 · 隐私账号与分板块可见性 · W3 单点登录 · 员工名单同步部门 · 13 个板块的管理后台与操作留痕 · 中英法三语

一句话：不是六个小站拼起来的 —— 一套账号、一套通知、一套评论、一套权限，加一个板块不用重建这四样。

功能面已经完整， 后面几页讲的是每个板块里真正难做、值得说的那部分。

【已有能力】后台架构：四层，每个方块都能指到具体代码

一份代码同时跑公网根路径与内网 /ai-community 子路径，零分支

接入层

浏览器 / CLI / nginx

nginx 反向代理

internal /_video/ 用内核 sendfile 直发字节

SSE 关闭缓冲

单请求体上限 5 GB

skills CLI（个人令牌）

systemd 托管

应用层

Next.js 单进程

72 个服务端渲染页面

151 个 API 路由

Auth.js 双通道登录（W3 + 密码）

统一权限闸门

配额与限流

出网网关（按主机分流）

三语 i18n

数据与存储

PostgreSQL + 本地磁盘

72 个数据模型 · 147 个索引 · 34 个已提交迁移

Serializable 事务 + 冲突重试

媒体走本地磁盘 + 一次性不可猜 key

ffmpeg faststart 重封装

外部依赖

全部可降级

内网大模型（直连，不走代理）

W3 UniPortal 单点登录

内网 SMTP 明文中继

企业代理（仅供知识库抓公网）

本机 whisper 语音转写

ffmpeg / ffprobe

三个关键工程决策：① 出网按主机分流而不是一个全局开关 —— 内网目的地直连、公网走代理，否则"开了代理 W3 登录挂，关了知识库抓不到文章"；② 鉴权归应用、发字节归 nginx，几十人同时拖进度条不会把 Node 单线程压满；③ 配置分级失败：安全项校验不过直接启动失败，装饰性参数填错回落默认值。

可自查：后台带三个诊断页 —— 网络出口、邮件、模型连通性，直接显示真实错误码和走的是代理还是直连，而不是一句"网络异常"。

可回归：26 个测试文件、290 条用例，全量跑完不到 1 秒且不依赖数据库。

这套架构的取舍主题只有一个：内网没有对象存储、没有 CDN、没有公网直连，能力都得自己长出来，而且要能降级。

【已有能力】技能市场：能搜、能试、能一行命令装走的内部包

社区里唯一一个直接进到别人生产流程里的模块

功能亮点

用户看到什么

- **一行命令装进项目。** `skills install <名称>` 默认装到当前项目，一次可装多个；合集包一条命令装齐一整套。
- **三档可见性 + 逐人审批。**受限 Skill 能被搜到但要作者点头；批准、拒绝、随时收回，每一步都有通知。
- **不用装就能先试。**详情页直接和"已经装上这个 Skill 的模型"对话，切走页面也不打断生成。
- **装了到底有什么不一样。**同一个任务、同一个模型跑两遍（带 / 不带 Skill），把两份真实输出交给 AI 写成对比报告。
- **Remix：**在别人的基础上改，编辑器里有逐行 diff，发布后自动记住来源，原作者的被 Remix 数 +1。

技术亮点

工程上难在哪

- **一个权限判定，六个出口复用。**访问判定压成一个纯函数（owner / 公开 / 已授权 / 需申请 / 拒绝.....七种结果），网页会话与 CLI 令牌先收敛成同一个"身份"再进它；下载、元信息、存储代理、文件浏览、更新检查、试用对话全走这一处。**"网页拦住了、CLI 没拦住"是这类平台最典型的漏洞。**拒绝一律返回 404 而不是 403，连"这个私有 Skill 存在"都不暴露。
- **下载配额与参数绕过。**按滚动 24 小时窗口对下载流水计数；**只信任地址里 install / update 两个来源值**，其余按浏览器标识回落 —— 否则手拼一个"试用"参数就能白嫖配额。绕开主路径直接取包的那条路也被堵上并同样计数。
- **撤权即断更、更新可回滚。**CLI 批量检查更新时复用同一套判定，授权被收回的 Skill 直接当"不存在"：既不提示更新，也不暴露它还在。客户端更新前先把目录改名备份，装失败自动回滚，绝不留下装了一半的 Skill。
- **内网没有 npm 源，CLI 自己做分发。**安装包挂在站点上，一条命令装好，升级是从当前服务器地址自举重装；地址三级解析（命令行 > 登录时存的配置 > 构建期内置），**同一个包既能指外网也能指内网。**

规模与口径：512 MB 单包上限 · 8 个 AI 辅助动作 · 试用上下文 150 KB / 实测双跑 64 KB · 试用限流 60 次/小时（匿名 10 次/小时/IP）· 合集包最多 200 个 Skill · 令牌 32 字节随机、只存哈希，管理员关掉"允许使用 CLI"即刻全部失效

这一页最值得讲的是 **"一处判定、六处复用"** —— 权限散在各处是内部平台最常见的事故来源。

【已有能力】知识库：AI 先替所有人读一遍，答案带原文出处

一份 60 页的报告，一个人读完，后面的人直接问

功能亮点

用户看到什么

- **贴个链接就入库。**公众号 / 博客链接，或上传 PDF、EPUB、PPT、Word，几秒出正文、封面、作者和预计阅读时长；重复提交自动识别是同一份。
- **AI 先通读：**整体导读（概述 / 脉络 / 要点 / 值得问的问题）+ 逐章摘要，中英双语，结果全站共享 —— 第二个人打开不再花一分钱模型调用。
- **问文档，答案带原文出处。**每句结论后跟一个可点角标，点一下跳回原文并高亮，能立刻验证 AI 有没有编。
- **划线、批注、共享笔记。**打开"公开我的笔记"后，同事读同一篇能看到你的划线，还能针对某条划线开讨论。
- **选中即译**（中英互译）· 读到第几章自动记住 · 书架、评分、评论。

技术亮点

工程上难在哪

- **五种格式收敛成一套坐标系。**网页 / PDF / EPUB / PPT / Word 各有抽取器，但都产出同一结构：净化后的章节 HTML + 纯文本 + 检索片段，并强制"片段文本必须等于章节文本的这一段"这条不变式 —— **正是它让 AI 的引用和用户手划的线落在同一个偏移空间里**，能互相跳转。扫描件和字体映射损坏的 PDF 主动识别并降级提示看原文，不把乱码喂给 AI。
- **逐章 AI 索引：断点续跑 + 抢锁 + 成本封顶。**结果直接写在章节行上当检查点，失败或重启只补没做的章；并发用一次原子更新抢锁、配 10 分钟过期防死锁；单篇最多索引 120 章封住成本。中文结果先落库再跑英文翻译 —— 翻译途中被重启不能把已跑完的中文一起丢掉。
- **没有向量库，就用两阶段检索。**第一阶段只让模型看"每章摘要 + 关键词"这份索引，选出候选章并给出会在原文里字面出现的检索词；第二阶段用这些词做数据库匹配。引用信息随流式响应的第一帧下发 —— 放响应头会撑爆 nginx 的缓冲区上限。
- **抓公网的安全边界：**链接抓取做 SSRF 防护并对**每一跳重定向**重新校验（只在内网部署放行内网地址）；EPUB 解压对单个条目卡上限防压缩炸弹；公众号图片会被重新托管到本地，否则防盗链让图全裂。

规模与口径：6 种入库来源 · 单篇最多 120 章 · 检索片段目标 500 token / 硬上限 800 · 单次问答上下文 10,000 token · 每人每篇最多 500 条划线 · 抓取总预算 90 秒、最多 5 次重定向 · 27 个前台 API 路由 · 阅读器 145 条三语文案

这个模块的含金量在 "AI 说的话能被一键核实"：引用不是装饰，是能点开跳回原文的坐标。

【已有能力】讨论区：轻的随手发，重的沉淀成帖

一个入口两种形态，不用维护两个系统

功能亮点

用户看到什么

- 上面是动态流，下面是论坛。随手发想法、贴图、贴文件走动态；要沉淀的开话题、按主题归档、可被检索。
- 一条动态能带 9 张图 + 1 个视频 + 4 个文档。视频卡片内直接播、能拖进度条；PPT / Word 点开就下载并保留中文原文件名。
- 六种表情反应（赞 / 精彩 / 支持 / 喜爱 / 有启发 / 有趣），点数字能看到"谁点了"。
- 两级评论 + 通知直达。别人回你时，通知点进去自动展开那一楼、滚到屏幕中间并闪一下高亮 —— 哪怕它在第三页还没加载。
- 话题可打多个主题标签，列表能看出"谁在聊、多少人看过"，浏览量按人按天去重。

技术亮点

工程上难在哪

- 两套翻页游标，按排序方式分流。"最新"用显式键集游标（把上一条的时间+id 编进游标）；**刻意没用框架自带游标 —— 那条帖子被删或被置顶后它会静默翻错页**。"热门"排序会随点赞实时漂移，没有稳定锚点，就退化成偏移游标，用简单换正确。
- 反应计数的语义边界。一人一帖只有一行记录，总赞数是所有表情之和，所以"从赞切换成有启发"只改那一行、完全不碰计数器；只有新增和取消才走事务同时改行和计数。并发冲突时整体回滚并返回权威真值，用户永远看不到报错，数字也不漂。
- 墓碑删除 + 守卫式事务。有回复的评论删成墓碑保住楼层结构；判断不能只靠事务外的预读，而是把条件直接写进更新语句的 WHERE，匹配不到就整体放弃 —— "两个人同时删"和"删的瞬间来了新回复"都不会把计数改花。论坛还多一步：删回复后从剩余回复重算最后活跃时间，堵住"回一条再自删把话题永久顶上去"。
- 附件的四道信任链。只认存储 key、地址服务端重新推导、正则限死前缀再做路径穿越校验，**再加一条：已经被别的帖子引用过的 key 直接拒绝**（key 在地址里肉眼可见，否则能把别人的文件挂到自己名下）；删帖回收磁盘按引用计数走。
- 中文文件名不能进响应头 —— 超出 ByteString 范围会让下载接口直接 500。这里按 RFC 双写：ASCII 兜底 + 百分号编码的真名。

规模与口径：10 张数据表 · 16 个 API 路由 · 3 次结构演进 · 单视频 1 GB / 单文档 100 MB / 每人每天 5 GB 上传预算 · 8 个主题分类（每话题最多 3 个）· 摘要按码位截断 140 字（不切断表情和生僻字）

这一页全是"写错了也不报错、只是数据悄悄不对"的坑：翻页会丢帖、计数会漂、浏览量能被伪造头刷上去。

【已有能力 · 最新上线】投票活动：把"群里接龙 + Excel 统计"变成一条正规评选流程

作品征集 → 审核 → 投票 → 榜单 → 明细导出，全程留痕

功能亮点

用户看到什么

- **两种办法建活动**：组织者批量上传作品（**按文件名规则自动解析出作品名 / 作者 / 工号**，手工改过的行下次重新解析不会被覆盖）；或者打开"成员自主投稿"，同事自己传、填表单、发起人审核后展示。
- **规则可配**：每人几票、按总量还是**按天**发票、单个作品最多投几票、允许不允许撤票改投、截止时间、结果什么时候公开。
- **匿名评选**：投票期只显示作品编号、不显示作者，结束后再公示 —— 评的是作品不是人。
- **作品顺序每个人看到的都不一样**（随机但对同一个人稳定），避免排前面的天然占便宜。
- **大图灯箱看作品、右侧可评论**；组织者随时看实时榜单、参与人数、每个作品是谁投的，一键导出 CSV 存档公示。

技术亮点

工程上难在哪

- **票数不是累加出来的，是每次从选票明细重算的**。投票在一个 Serializable 事务里完成：写选票 → 从明细重算这个作品的票数 → 重算活动总票数和参与人数。**改票、撤票、并发投票都不可能让票数对不上**。每一票都要改同一行活动记录，高并发下必然冲突，所以配了带抖动的退避重试（最多 5 次），而不是硬撞。
- **预算与上限在事务里判定**。"每人 10 票""单作品最多 1 票"这类规则如果在事务外查，两个标签页同时点就能超发；这里是进事务后重新统计已用票数再决定。按天发票的桶用北京时间的日期做键，跨时区不会串天。
- **刷票的三道门**：内网部署下投票**强制要求 W3 实名身份**（密码注册的小号直接拒绝）；一人一作品一天只有一行记录，物理上无法重复；下架的作品不再接受新票，但**撤票永远可用**——否则你的票会被永久占住。
- **结果可见性是服务端裁剪**。没到公开时间时，接口返回的票数字段直接是 null，不是前端藏起来 —— F12 也看不到。
- **导出的 CSV 带 BOM**（Excel 打开中文不乱码），并对以 = + - @ 开头的用户文本加引号前缀，**防止表格公式注入**；排名用并列名次算法，和页面上看到的完全一致。

规模与口径：4 张新表 + 4 次迁移（8 月 17 日）· 14 个 API 路由约 2000 行 · 投票限流 60 次/分钟 · 冲突重试最多 5 次 · 作品图片 / 视频走登录 + 不可猜 key 的媒体路由（支持 Range 拖动）· 自定义投稿表单字段 + 投稿须知 + 驳回理由

这是最近上线的功能，也是最能讲"并发下票数为什么不会错"的一页 —— 票数每次都从明细重算，不做加减法。

【已有能力】帖内投票与表情包：一次做进编辑器，19 个地方立刻都能用

和上一页的评选活动是两件事：这个是"随手在正文里插一个投票"

功能亮点

用户看到什么

- **任何输入框都能发起投票。**编辑器工具栏点  填题目和选项，直接插进正文 —— 动态、评论、讨论帖、活动介绍、视频简介、Skill 说明等 **19 个能写字的地方**全部自带，不用各自开发。
- **规则可配：**单选 / 多选（可限最多选 N 项）、匿名、投票后才看结果、截止时间；非匿名投票在选项下显示投票人头像。
- **能改票、能撤票、能提前结束；**未登录也能看到题目和公开结果，点投票才提示登录并带回原页面。
- **微信式个人表情包。** 打开个人表情库（最近 / 全部 / 收藏），支持 GIF 动图原样保存；看到别人发的表情，点开大图或右键就能一键收藏。

技术亮点

工程上难在哪

- **投票不改任何一张业务表：**正文里只存一行 `[poll:id]` 纯文本，渲染时由一个 84 行、零依赖的模块切成"文字段 + 投票段"。难在边界：代码块里的示例必须保持惰性文本、行内提及不能触发、4 空格缩进按规范算代码块也要放行。**一条契约三处复用**（渲染器 / 编辑器 / 列表摘要），36 个用到 Markdown 渲染的页面零改动全部支持。
- **重算式投票事务。**不用加减计数：删旧票 → 写新票 → 从明细重算每个选项票数和去重人数，全在 Serializable 事务里，写冲突最多重试 3 次，仍失败明确报错绝不当成功。数据库主键再兜一层，同一人同一选项物理上无法重复。
- **"投票后才可见"是真的服务端裁剪：**没投的人拿到的 JSON 里票数直接是 null。还堵了一个隐蔽漏洞 —— 这类投票**禁止清空式撤票**，否则"随便投一票看结果再撤回"就是免费偷看；改票依然允许。
- **编辑器自定义节点的三个坑：**初次加载不触发任何事务（要在插件构造时补一次规范化）、规范化必须跳过撤销重做、插入位置强制提升到文档顶层（光标在引用块里时 token 会被嵌进去，渲染端永远匹配不到，变成看不见的孤儿投票）。这些用无头冒烟测试钉死，防框架升级悄悄改行为。

规模与口径：4 张表 · 19 个编辑器接入点 · 36 个页面复用同一渲染器 · 单条内容最多 10 个投票 · 选项 2-12 个 · 表情包每人 500 张、单张 10 MB（GIF 不重编码）· 20 条自动化用例覆盖切分与编辑器节点 · 三语文案 74 条

值得讲的是 **"把产品问题降成一行文本"**：投票没有侵入任何业务表，却让全站 19 个输入框同时具备了这个能力。

【已有能力】极客视频 + 随刷短视频：一套自托管底座，全程不出内网

长视频要沉淀，短视频要传播 —— 同一套数据，两种消费方式

功能亮点

用户看到什么

- **打开就能刷的竖屏流。**一屏一条、快甩也不会跳过；双击点赞、拖进度条、键盘上下切换、空格暂停。
- **人人都能发短视频。**拖个文件进来自动读时长、抓封面帧，写句文案点"AI 润色"就能发。
- **自动生成中英双语字幕。**发布后服务器在后台把语音转成字幕再翻译成另一种语言，播放时一个按钮在中 / EN / 关之间切，选择会被记住。
- **长视频有 AI 看点摘要，还能就这条视频提问，**答案只依据这条视频的文字稿，答不上来会直说。
- **搬运必须署名：**上传时选自创还是搬运，选搬运必须填原始链接和原作者，播放页显示可点来源。

技术亮点

工程上难在哪

- **鉴权归 Node，字节归 nginx。**文件路由只做登录校验和路径穿越检查，然后返回一个内部重定向头，让 nginx 用内核 sendfile 直接吐字节 —— 否则几十个人同时拖进度条，所有字节都要挤单线程。没开这个开关时保留自己实现的 Range 分片（206 / 416 / content-range 全部自己算）作为等价兜底。
- **只在当前条 ± 2 挂真正的播放器。**每个可播放元素都占一个硬件解码器，移动端解码器池只有个位数，一个无限流全挂真元素必崩黑屏；其余位置渲染同高度的占位保住滚动吸附几何，离开视口主动释放解码器。判活只用一个观察器取交叉比最大者，各自判断会出现两条同时播放。
- **静音优先播放与失败分类。**浏览器只允许静音自动播放，所以先静音起播、用户开声后记住偏好；播放的 Promise 必须接住并分类 —— 切源竞态的中断要忽略，只有权限被拒才切成"点一下播放"的兜底 UI。
- **离线字幕流水线。**内网调不到云端语音识别，整条链路本地化：抽 16 kHz 音频 → 本机 whisper 转写 → 家用大模型按 30 条一批翻译，编号对不上就整批放弃（**宁可只出原文，也不出错轴字幕**）。任务用一次原子更新占位，重复触发不会跑两遍。字幕不用浏览器原生渲染 —— 竖屏下会画到黑边里跑出可视区，所以自己画。
- **上传回显 key 的三重复验：**正则查形状、磁盘确认存在（体积一律以服务端为准）、再查有没有被别的视频占用（否则能把别人的文件挂到自己名下）；时长以服务端探测为准。

规模与口径：同时最多 5 个真播放器（当前 ± 2 ）· 每页 8 条、剩 3 条预取 · 累计 2 秒才算一次观看，同一人同一条每天只计 1 次 · 长视频上限 5 GB · faststart 重封装适用 ≤ 2 GB · 转写超时 90 分钟 · 19 个 API 路由 / 9 张表 / 约 6900 行代码

短视频不是新表，是复用长视频的模型加一个标记 —— **省下一半工作量，代价是每个视频查询都必须显式表态，这条契约写进了代码约定。**

【已有能力】社区活动：跨时区的日历，报名即订阅提醒

一场讲座不再只到一个邮件列表

功能亮点

用户看到什么

- **时间按你所在地显示。**组织者按自己的时区填，别人看到的是自己本地时刻，两地不同时补一行"北京 14:00"，不用心算时差。
- **一键报名，开场前约 30 分钟自动提醒**（站内 + 邮件），带时间、地点和线上会议链接；取消参加就不再提醒。
- **日历和筛选联动：**右侧月历上有活动的日子带圆点，点某天列表只留那天；按大类 / 主题 / 城市 / 形式筛选时圆点和数量跟着变。
- **加进自己的日历：**Google 链接或下载 .ics 给 Outlook / Apple；活动被取消后再导出，标题自动带【已取消】前缀。
- **「我参加的」独立视图，**封面可以拖着取景，一张竖图也不会被拦腰截断。

技术亮点

工程上难在哪

- **真实 UTC 瞬时 + 组织者时区，不存"本地时间字符串"。**换算完全用浏览器内置的国际化能力实现，不引第三方时区库：先按猜测偏移算一遍再用结果二次求解，**夏令时切换日也能拿到正确瞬时**（多伦多 8 月填 14:00 → 18:00Z，1 月填 14:00 → 19:00Z，有测试钉住）。全天活动是另一个域：只存日期、永不换算，免得"9 月 3 日全天"在别的时区变成 9 月 2 日。
- **封闭时区集合编译成精确 SQL。**"即将举行 / 某一天"本质是墙上日期条件，同一个日期在不同时区是不同的 UTC 区间。因为时区只有四个选项，条件被展开成"4 个时区分支 + 1 个全天分支"下推给数据库 —— 没有近似、也不是捞回内存再过滤，筛选、计数、月历共用同一套条件。把产品约束变成了性能与正确性的杠杆。
- **零 hydration 不一致的双时区渲染：**服务端和首帧都确定性地按"活动自身时区"渲染，页面激活后再读浏览器时区切换 —— 直接渲染本地时间必然报错闪烁。
- **报名与提醒用了两种并发原语。**计数用守卫式事务，双标签页同时点冲突的那次整体回滚再读回真值（不 500、不漂）；提醒用原子认领 —— 把"已提醒时间"从空改成现在，返回 0 行说明别的进程抢到了。**所以"网页轮询顺带扫"和"服务器定时任务"两条触发路径可以同时开，物理上不可能重复发同一条提醒**（内网没有调度基础设施，这是刻意的双保险）。
- **会议链接是服务端裁剪：**未登录访客拿到的 JSON 里根本没有这个字段；.ics 导出也不含它，全天活动的结束日按规范多加一天，否则各家日历会少显示一天。

规模与口径：4 个固定时区 / 7 个城市 / 4 个大类 / 10 个主题（每场最多 3 个）· 1 个日期筛选 = 5 个 SQL 分支 · 提醒窗口开场前 35 分钟、每轮最多处理 50 场 · 建议定时任务每 5 分钟一次 · 月历一次最多 400 条 · 模块约 4500 行 · 三语文案 627 条

"按你所在时区显示"听起来是小功能，**实际是一整套时间建模：**存什么、按什么筛、怎么渲染，三处都要一致才不出错。

【已有能力】平台层：一个铃铛收全站，%K 搜全站

这一层决定了"再加一个板块"要花多少功夫

功能亮点	用户看到什么	技术亮点	工程上难在哪
■ 一个铃铛收全站消息：视频评论、动态与论坛回复、知识库批注、Skill 下载审批、活动开场提醒，全汇到导航栏同一处。 ■ 点通知直达那条回复，不是跳到页面顶部 —— 滚到那条评论、高亮 2.5 秒，必要时自动展开它所在的楼层。 ■ 通知开关自己定：四类事件 × 站内信 / 邮件两个渠道，互不影响。 ■ %K 一个框搜全站：Skill、知识库、讨论、视频、活动、投票、合集包、反馈、用户、分类、标签 11 类分组展示。 ■ 需求板：任何人提建议、给别人的条目 +1、二级讨论，条目挂"待处理 / 已计划 / 处理中 / 已完成 / 不采纳"状态。		■ 通知总线：失败绝不回连主写入。通知从业务写入点发出，所以它的失败不能把主写入带崩 —— 三层降级：查偏好失败回落默认值、写站内信单独兜底、每个入口整体兜底，调用侧一律不等待。正因为总线自己扛住所有异常，后面接入的 6 个业务域都只加了两三行调用，没有一个要写补偿逻辑。 ■ 公告扇出：批量写 + 跃迁去重。一次性把用户连同偏好取出（避免每人一次查询），站内信攒成一次批量插入。关键防呆在编辑接口：只有"未发布 → 已发布"这一次跃迁才扇出，之后改错别字再保存也不会二次轰炸全员。 ■ 深链聚焦：目标可能还没加载。评论分页 + 回复懒展开，通知里指向的那条可能在第三页某个楼层里。做法是先请求一个解析接口把它归位到所属楼层、必要时把该楼插到列表头部，再由评论树自己完成展开、滚动、高亮 —— 而不是轮询 DOM 碰运气。 ■ 内网明文 SMTP 适配 + 可自查诊断。默认的加密协商打内网中继会静默卡在握手手上，所以显式关掉升级并加三档超时；业务路径吞掉发信错误（不能让发信失败弄崩主流程），另留一条会抛错的诊断通道给管理员看真实原因。 ■ 全站搜索把可见性写进查询本身：12 张表并行匹配，每类自带各自的可见性谓词（已发布 / 未删除 / 非私有 / 账号有效.....），私密账号的工号在服务端就被抹成空串。	

规模与口径：6 种通知类型 · 12 个通知入口被 11 处业务调用 · 4 类 × 2 渠道 = 8 个开关 · 8 套邮件模板 · 铃铛 60 秒轮询 · 搜索并行 12 条查询归成 11 组 · 搜索词硬截断 64 字符 · 反馈板 5 种状态 / 3 种分类

判断一个内部平台成不成熟，看的是 "加第七个板块时，通知、搜索、权限要不要重写" —— 这里答案是不用。

【已有能力】身份与权限：W3 一键登录，谁能看到什么在服务端裁剪

实名社区里，这一层的每个决定都直接关系到能不能过安全评审

功能亮点

用户看到什么

- **W3 一键登录，老账号自动合并。**点"华为账号登录"跳 UniPortal，回来就是注册用户；之前用邮箱注册过的会合并成同一个人，不会变两个账号。
- **部门 / 研究所自动挂上。**登录时按工号匹配员工名单，名字旁边自动出现「XX部 · XX所」，用户自己不用填也改不了。
- **隐私账号一键开关：**打开后别人看你只剩昵称和头像；个人主页还能按六个板块分别开关，关掉的板块别人连数据都拿不到。
- **内容三档可见性 + 申请制：**公开 / 受限（能搜到、下载要批） / 私有（对别人直接 404，连存在都不暴露）。
- **管理后台一站式：**13 个板块，用户权限开关即时生效，员工名单支持粘贴 / CSV / XLSX 导入并显示逐行失败原因，所有管理动作可按动作、目标、管理员检索。

技术亮点

工程上难在哪

- **子路径部署下的登录双向修补。**内网挂在共享域名的 /ai-community 下，框架会把这段前缀从入站请求里剥掉，可 OAuth 回调地址又必须带着它 —— 两个方向要求相反，结果就是登录直接失败。解法是出站把前缀拼进去、入站再加回来、客户端会话与回跳地址 pin 同一个变量，再给浏览器的 fetch 打一次全局补丁让 50 多处调用自动带前缀（而不是逐个改，新代码也不会忘）。**四处任缺其一都登不上**，而且根路径部署时全部是空操作。
- **华为非标 OAuth 的适配层。**对方实现与通用约定有五处不一致（token 交换要 JSON、不给 token_type、过期时间可能是 13 位绝对毫秒、用户信息要 POST、出错时经常是 HTTP 200 带错误码）。做法是只拦截两个端点做协议整形，其余交给框架；这个模块写成无环境依赖的纯函数以便单测。
- **首登幂等：**回调可能并发触发两次（双击、第二个标签页、网关预取），朴素的"先查再建"会让输家撞唯一索引 —— 表现就是**新人第一次登录 500、刷新一下又好了**这种最难复现的 bug。这里靠唯一索引冲突后重读兜住。
- **隐私是服务端裁剪，不是前端隐藏。**每个服务端边界都必须过统一的身份裁剪函数，它返回一个全新对象，多查出来的字段根本不会漏出去；单测直接锁死输出对象的键集合。**98 处调用、39 个文件一致执行。**
- **员工名单只认工号不认用户名。**按用户名匹配更省事，但外网是开放注册，用「工号@任意域名」注册就能继承别人的部门并反查花名册 —— 所以匹配键只用一次完整 W3 登录才能写入的工号；工号入库统一转小写，否则大小写不同会绕过唯一约束产生两行。
- **鉴权位置的反直觉选择：**后台拦截通常放在边缘中间件里最省事，但在反向代理 + 子路径的组合下它读不到安全会话 cookie，会把已登录的管理员判成未登录 —— 所以刻意**没有中间件**，网关放在服务端布局里，20 个后台 API 再各自独立校验一次。

规模与口径：20 个后台 API 每个独立校验 · 151 个路由中 74 个含权限判断 · 98 处身份裁剪调用分布在 39 个文件 · 37 种管理动作写入操作日志（44 处调用） · 员工名单单次导入上限 2 万行 · 密码哈希 bcrypt cost 12 · 3 种账号状态（仅密码 / 仅 W3 / 两者）

这一页可以只讲一句：**隐私功能的含金量不在开关有多少个，而在数据到底有没有离开服务器。**

【方向】平台已经能用，但"活"不起来 —— 先定路线，再谈功能

同样一个平台，往三个方向长，会长成完全不同的东西

现在的样子：东西都在，人不在。发东西的永远是那几个，大多数人一年打开两次 —— 一次是找某个 Skill，一次是别人转了个链接给他。

为什么不是"再加功能"：缺的不是入口，是发的动力和看的理由。功能加到第七个，没人发还是没人看。所以先定路线。

A 真人社区 主推

让每个人都愿意发一条，同事之间先认识起来。

不端着：调通了什么、踩了什么坑、一张截图、一个吐槽都算内容。围绕话题聊起来，发得好的人被看见、成为社区里的红人。

再配几个人负责维护和组织新鲜内容。

押注：人 · 关系 · 日常

B 专家专栏

少数专家产出高质量长文，其他人来读。

对标 2012 网站、稼先社区：约稿制、专人维护、内容严肃、有编辑把关。

做成了是资产，做不成是断更的空壳。

押注：内容含金量

C 资源挂钩

做任务、发帖换 AI 额度，用资源把人拉进来。

覆盖面最广、见效最快 —— 只要 AI 额度是稀缺的，人就一定会来。

但来的人是冲额度来的，不是冲社区来的。

押注：资源杠杆

三条路不是三选一。下一页把利弊摊开对比，结论是：A 做主线，B 做栏目，C 做放大器。

【方向】三条路线摊开比：谁供内容、多少运营、什么风险

同一张表看完再决定，避免"哪个都想要"

维度	A · 真人社区（主推）	B · 专家专栏	C · 资源挂钩
谁来供内容	全员 ，日常也算内容 —— 供给面最宽	少数专家，约稿制 —— 供给面最窄	被动产生，人人为了额度而发
见效速度	中：4—8 周能看出来	慢：内容生产周期以月计	最快 ：上线当天就有人来
运营投入	中：3—5 个种子活跃者 + 每周一次精选	高 ：要编辑、要约稿、要排期，断更即死	低：规则自动跑，但要额度池的授权
最大风险	内容参差；实名环境下有人不敢发第一条	作者池不够就断更；和 2012 / 稼先 同质化 ，我们没有他们的作者储备	为拿额度灌水 ，质量崩坏；额度一停人就走；把"分享"变成"交易"
怎么防	轻量分区（日常 / 技术 / 求助）+ 只做正向激励 + 不设全站总分榜 + 管理员带头发	不单独立项，降级成社区里的一个 专栏栏目 ，有稿就发，没稿不塌	额度只奖励 被别人用到的 贡献（被采纳 / 被安装 / 被致谢），不奖励发帖条数
我们的独有优势	实名 + 部门信息 ：让"认识同事"这件事成立，外部社区花钱也做不到；同事关系本身就是分发网络	内网可以放不能外发的内容	内网模型免费，额度这张牌 只有我们打得出
结论	★ 作为主线 ：先把人和关系立起来	作为 A 里的一个栏目：成本可控，好稿照样有位置	作为 A 的放大器：用额度奖励优质贡献，不用它当主线

选 A 的真正理由：**B 和 C 都能被 A 吸收，反过来不行** —— 有了人，专栏和额度才有落点；只有专栏或只有额度，没人也留不住。

【方向】主推路线 A 怎么落地：让人可见 → 让发容易 → 让留有理由

三步，每步都有对应的功能和运营动作，见第三章

① 先让"人"可见

问题：现在打开社区看到的是内容列表，不是人。

做什么：关注与动态流、个人名片、话题页、@ 提及、本周红人榜（按被别人消费的互动排，不按发帖量）。

约 4 周 · 见第 19 — 21 页

② 再让"发"变容易

问题：第一条最难发 —— 怕写不好、不知道发哪。

做什么：一键发日常（模板 + 话题引导）、AI 帮你把草稿理顺（只润色不代写）、新人首帖引导与欢迎回帖。

约 8 周 · 见第 22 — 23 页

③ 最后让"留"有理由

问题：热情烧完就没人再发。

做什么：贡献档案与成就称号、精华与专栏、AI 额度奖励挂到"被用到"的贡献上、线下活动与社区联动。

按季度 · 见第 25 页

配套的运营动作

不是代码，但决定成败

- 3—5 个种子活跃者，首月每人每周发 2 条，把"日常也能发"这件事演示出来。
- 每周一次精选 + 红人榜，10 分钟的活，但这是"发了有人看"的唯一证据。
- 专人组织新鲜内容：每两周一个话题（新模型实测、踩坑合集、工具比拼），有人起头就有人接。
- 管理员带头回帖：新人第一条帖 24 小时内必须有人回 —— 这条比任何功能都管用。

怎么算成功

四个数，每月看一次

- 周活跃发帖人数 —— 北极星，不看 PV
- 首次发帖转化率 —— 注册过的人里有多少发过第一条
- 帖均互动人数 —— 发出去有没有人理
- 跨部门互动比例 —— 有没有真的"认识到别的组的人"

这三步的顺序不能换：人不可见的时候做激励，等于给空房间发奖品。

【方向】一主两辅：专栏当栏目，额度当放大器

不砍掉 B 和 C，但都不给它们独立立项

B · 专家专栏怎么落

降级成栏目，不做独立平台

- 形态：社区里的一个「专栏」标签 + 首页固定一个位置，不做单独的站。
- 供给：约稿 + 自荐，编辑（管理员）择优挂精华；有稿就发，没稿不塌 —— 这是它和独立立项最大的区别。
- 沉淀：专栏文章自动进知识库，可以被 AI 检索、被引用，不只是一篇帖子。
- 作者激励：作者认领 + 被引用计数 + 荣誉称号，比稿费更适合工程师。
- 和 2012 / 稼先的区别：不比谁的文章更权威，比谁离你的工作更近 —— 内网模型怎么用、我们自己的坑怎么填。

C · AI 额度怎么用

当放大器，不当发动机

- 只奖励被别人用到的贡献 —— 回答被采纳、Skill 被安装、文档被收藏、收到致谢；**发帖条数一分不给**。这一条决定了它不会变成灌水机。
- 额度是加发，不是门槛 —— 基础额度谁都有，贡献换的是更高的上限；新人和只看不发的人不会被卡。
- 发放自动化：按等级和成就自动加，不需要人工审批，运营成本接近零。
- 需要一次授权：额度池归谁、一个季度多少，要提前定 —— 这是第三章里唯一需要外部批的东西。
- 止损：一旦发现“为拿额度灌水”，先降权再停发，规则公开可查。

一句话记住：A 是人，B 是内容，C 是燃料。燃料浇在没人的房子上只会烧掉预算。

【新功能】十二项，按"让人可见 → 让发容易 → 让留有理由"排

每一项都写清楚它服务于哪一步；后面逐个展开功能与技术

#	功能	解决什么问题	服务于哪一步	工作量	批次
1	关注与动态流 + @提及	打开社区看到的是内容列表，不是人；发了没人知道	让人可见	中	第一批
2	个人名片与本周红人	不知道谁是谁、谁在做什么；发得好的人没有出口	让人可见	小	第一批
3	统一话题	论坛分类 / 知识库分类 / 活动主题三套割裂，同一个话题散在各处	让人可见	中	第一批
4	一键发日常 + AI 顺稿	第一条最难发：怕写不好、不知道发哪、排版麻烦	让发容易	小	第一批
5	新人首帖引导与欢迎	发出去零回复，第一次就是最后一次	让发容易	小	第一批
6	一页速递 + 每周精选	发了就沉，只有当天在线的人看得见	让发容易	小	第一批
7	问答终态（采纳）	讨论永远发散，半年后同样的问题再问一遍	让留有理由	小	第二批
8	AI 值班员	新帖零回复的死寂没人管	让留有理由	中	第二批
9	贡献激励 + AI 额度	帮了人的人得不到回报；热情烧完就没人再发	让留有理由	中	第二批
10	专家专栏	高质量长文没有固定位置，写完就沉	路线 B 的落点	小	第二批
11	语义搜索地基	搜不到就等于没有；重复提问、重复上传	基础设施	大	第二批
12	MCP 出口 + 双周挑战赛	知识只能给人读，不能给 agent 用；潜水的人缺一个参与钩子	差异化 / 运营	中	第三批

第一批六项全是小到中的工作量、零外部依赖，目标只有一个：让"发一条"这件事有回响。

【新功能 1】关注与动态流 + @提及：把"内容列表"变成"人的现场"

这是路线 A 的骨架，其余社区功能都挂在它上面

功能：用户看到什么

让人可见 · 第一步

- **关注人、关注话题。**首页多一个「关注」流：你关注的同事发了什么、参与了什么讨论、上传了什么 Skill，一屏看完。
- **发现页推荐关注。**同部门的人、和你装过同一个 Skill 的人、你读过的文档的作者 —— 按"你可能认识"排。
- **@ 提及。**发帖和评论里 @ 同事，对方收到通知并深链到那一楼；跨部门 @ 也能到。
- **互相关注的人在名片上显示**「你们都关注了 X」，让"认识"这件事有起点。
- **关注是双向可见的轻社交，不做私信** —— 内网实名环境里，公开可见比私聊更安全。

技术：怎么做的

零新外部依赖

- **关注关系用复合主键连接表** (followerId + followingId)，和现有的报名、点赞是同一套房规，天然去重、天然幂等。
- **动态流走"读时聚合"而不是写时扩散。**内网量级下每人关注数有限，读时按关注列表拉取比给每个粉丝写一份收件箱简单一个量级，也不会有一致性问题；分页直接复用现有的 `createdAt | id` 键集游标，删帖不会让游标错位。
- **@ 提及复用现有通知管线**，新增一种通知类型要同时改枚举、用户偏好开关和默认值三处 —— 这是仓库里定好的契约，漏一处就会出现"关不掉的通知"。
- **@ 的解析在服务端做：**只认真实存在且可见的用户，防止用 @ 探测隐私账号；私密账号被 @ 时不暴露部门信息，走统一的服务端身份裁剪。
- **推荐关注是纯 SQL：**同部门 + 共同安装 + 共同阅读三路取交集，不引入推荐系统，先跑起来再说。

关注流是整条路线的地基：没有它，"红人""话题""精选"都无处可去。

【新功能 2】个人名片与本周红人：让发得好的人被看见

"成为红人"是路线 A 最直接的动力，但排法决定了它是良性还是灾难

功能：用户看到什么

让人可见 · 第一步

- **个人主页升级成名片**：头像、部门、在做的方向、最近产出（Skill / 文档 / 帖子 / 讲过的活动）、收到的感谢，一屏认识一个人。
- **本周红人榜**放在首页侧栏：本周作品被最多人用到的几位，配一句"因为什么上榜"（被 8 个部门装了 Skill / 一条回答被采纳 12 次）。
- **新人保底线**。榜单固定留一个位置给本周第一次发内容的人，随机轮播 —— 内部小社区最怕永远同几个人霸榜。
- **名片可以关掉**。隐私开关按板块控制，不想露的人整体隐藏，功能照常用。

技术：怎么做的

复用现有身份契约

- **上榜看"被消费"，不看"发了多少"**。排序输入只有被安装、被采纳、被引用、被感谢这几类别人产生的信号，自己发多少条一分不加 —— 这一条决定了榜单刷不动。
- **计分内核四条防刷规则**全站共用：每类行为按日封顶、同类重复按 $1/\sqrt{N}$ 递减、跨部门受益加权、只认被消费的贡献。做成一个共享库，等级、成就、红人榜、悬赏路由都调它。
- **新人保底轮播是纯 SQL**：按"用户 id + 周数"生成稳定随机种子，同一个人一周内看到的顺序不变，不同人看到的不同，零依赖、零状态。
- **渲染严格走服务端身份裁剪**：私密账号的部门 / 研究所在**服务端**就被摘掉，不是前端隐藏 —— 数据根本不会发到浏览器。
- **没有全站总分排行榜**。只有单项事实榜（本周被安装最多的 Skill、本周被采纳最多的回答），不把人压成一个分数排队。

红人机制的成败全在排序口径：**按发帖量排，一周就烂；按被别人用到排，刷不动也不伤人。**

【新功能 3】统一话题：把散在四处的同一个话题收到一页

现在论坛分类、知识库分类、活动主题是三套互不相通的标签

功能：用户看到什么

让人可见 · 第一步

- 一个话题一页。点开「RAG 检索」，同一页里是：相关讨论、相关文档、相关 Skill、相关活动、这个方向上活跃的人。
- 话题可以关注，进你的关注流；也可以在发帖时挂话题，发完就自动出现在话题页。
- 话题页有"这个方向谁最懂"：按被采纳、被安装、讲过的活动排，点开每条证据都能核实。
- 话题由管理员维护一份主表，用户只能挂已有话题，避免出现十七个写法不同的同义标签。

技术：怎么做的

晚做一天多一天债

- 做成映射层，不动历史数据。现有三套分类（论坛 categories、知识库细分类、活动主题）各自保留，另建一张话题主表 + 一张"话题 ↔ 内容"的反向索引表，写入时同步、存量夜间回填。历史内容不迁移、不改结构，随时可回退。
- 跨类型聚合复用全站搜索的既有查询层，那里已经按类型分别处理了可见性；话题页只是换了一个过滤条件，不新写一套权限判断 —— 权限最怕的就是两套实现。
- 可见性在服务端逐类型过滤：受限的文档、私有的 Skill、私密账号的帖子不进话题页，且是查询阶段就排除，不是查出来再筛。
- 这一层也是后面语义搜索的落点：先把"同一个话题"的关系用确定性的映射表建起来，再叠语义相似，比一上来就全靠向量稳得多。

三套标签各自长大以后再合并会很痛，趁现在体量小，先把话题这层公共地基铺了。

【新功能 4 / 5】一键发日常 + AI 顺稿 + 新人首帖引导

决定成败的不是编辑器有多强，是第一条帖子有多好发、发完有没有人理

功能：用户看到什么

让发容易 · 第二步

- **发帖框给模板，不给空白页。**「今天试了什么」「踩了个坑」「安利一个工具」「想问一句」四个模板，点一下就有骨架，配图直接粘。
- **AI 顺稿：**写完点一下，AI 把口语理顺、补个标题、建议挂哪个话题 —— **只给建议，永远不替你发**，你可以一键还原原文。
- **不必严肃。**日常、吐槽、一张截图都算内容，和技术帖分区不分级。
- **新人第一条帖打标，**进管理员和种子用户的"待欢迎"列表，承诺 24 小时内有人回。
- **发完给正反馈：**多少人看了、谁点了反应、被谁转到了哪个话题。

技术：怎么做的

复用现有编辑器与 AI 管线

- **编辑器不重做。**模板只是给现有富文本编辑器灌初始内容 + 预挂话题，图片、视频、文件、表情包、投票这些能力已经在里面了。
- **AI 顺稿是服务端调用、结构化返回。**沿用仓库里已经踩过坑的那套：思考型模型的思考块要过滤、JSON 提取要能处理截断和代码块，返回失败就静默降级成"不改"，绝不把半截内容写进正文。
- **永不自动写入。**AI 的结果放在对照面板里，用户点"采用"才进正文 —— 这是全站 AI 的统一红线：只建议、不落笔、可撤销。
- **"待欢迎"是一条 SQL 查询**（首帖 + 无回复 + 超过 N 小时），不需要新表；欢迎回帖到位后自动出列表。
- **阅读数按天去重，**复用讨论区已有的浏览统计口径，匿名用户按代理链最后一跳记 —— 第一跳可伪造，这个坑仓库里已经踩过。

一句话：把"发"的成本压到一分钟，把"发完没人理"的概率压到零。这两件事做成了，社区就活了一半。

【新功能 6】一页速递 + 每周精选：让好内容主动找到人

社区里最贵的不是内容，是"有人看见"

功能：用户看到什么

让发容易 · 第二步

- **每周一早上一页纸**：本周精选帖、一条技巧、一个踩坑、新上架的 Skill 与文档、本周红人，10 秒扫完。
- **七天没来的人收到个人版**：你关注的人发了什么、你订阅的 Skill 更新了、你书架上的文档有了新批注。
- **每周精选**由管理员从本周内容里挑 3—5 条挂标，作者得到「精华」标识和首页位置。
- **站内也有一份**：不看邮件的人在首页顶部也能看到本周这一页。

技术：怎么做的

一个定时任务的事

- **先用数据库把素材聚好，AI 只做编辑**。选题、排序、去重全是 SQL；大模型只负责把条目串成人话，提示词里硬性规定"没有链接支撑的话不许出现"——从机制上杜绝编造，这是对工程师人群唯一可信的 AI 周刊做法。
- **定时任务照现有的活动提醒那套写**：用一次原子更新认领待发送的行，多进程并发也不会重复发——这个模式仓库里已经在跑，直接抄。
- **草稿态兜底**：周刊先落成公告草稿，管理员十分钟定稿；24 小时没人动就按原样发出去，不卡在运营手上。
- **配套一起上**：发送台账、一键退订、回站点击埋点。内网邮件客户端拦外链图片，打开率不可信，所以只统计点击——指标口径要在功能上线的同一天定死。
- **缺席者判定看最后活跃时间**，活跃用户永远收不到打扰邮件。

速递是所有功能的分发器：**没有固定栏位，精选、话题、红人榜做出来也没人知道。**

【新功能 7 / 8】问答终态 + AI 值班员：让每个提问都有回响

零回复是社区最快的死法，而它恰好是 AI 最能帮上忙的地方

功能：用户看到什么

让留有理由 · 第三步

- 论坛多一种「问答」帖：作者可以采纳一条回答，帖子打绿勾；列表能筛「还没解决的」，7 天没采纳自动提醒作者。
- 24 小时内必有第一条回应。没人回的问答帖，AI 用站内已有的文档和已解决问答草拟一条答案，**每句话都带能点开的原文引用**，并且显著标注是 AI 写的。
- 专家一键核验。对口的人点"确认"或"纠正"，通过后挂「专家已验证」；任何人都能一键折叠 AI 的回复。
- 已验证的答案自动成为语料，下次 AI 答得更准 —— 越用越准的正循环。

技术：怎么做的

复用知识库 AI 管线

- 采纳只加三个字段（帖子类型、被采纳的回答、解决时间），计数更新走仓库里已有的守卫式事务写法：不做"先查后写"，避免并发下计数漂移。
- AI 首答复用知识库的检索管线：两阶段检索 + [cX-Y] 引用格式 + 流式前端都是现成的。但要如实说：现有检索是单篇文档内的关键词命中，扩到全站需要补一层全站分片与索引，也就是第 11 项的语义地基。
- 四条硬规矩随第一版一起上：只在语料够厚的分类开启、依据不足就不发言、任何人可折叠、7 天没人核验自动降级为"仅供参考"。
- 按提问人的权限检索 —— 受限文档不会出现在他看到的引用里，出口给 agent 时同样按权限裁剪。这是这个功能能不能过信息安全那一关的关键。
- 24 小时未答的扫描同样用原子认领，和活动提醒共用一套定时框架。

已采纳 + 专家核验过答案，就是内部 AI 最高置信的语料 —— Stack Overflow 2025 年 11 月把企业版更名 Stack Internal，主打的正是这件事。

【新功能 9】贡献激励：等级、成就、称号、精华，以及和 AI 额度挂钩

这是路线 C 的落点 —— 用资源奖励贡献，但绝不奖励发帖条数

成长等级 只升不降	L1 访客 → L5 社区专家。经验只来自 被别人用到的贡献 （被安装、被采纳、被引用、被感谢），发帖数本身不加分。每级解锁的是增量权益： 更高的 AI 调用额度 、更高的下载与上传配额、发起悬赏、创建合集。基础额度谁都有，不贡献也不会被卡。
成就 做成了什么才有	一次性里程碑，挂在名片上的成就柜：第一次发帖 · 第一次被采纳 · 被 10 个部门装过 · 办成一场活动 · 连续 4 周有贡献。 不设"连续登录""每日签到"这类只跟出勤有关的成就 —— 那种东西在研发人群里只会招人烦。
称号 会过期	领域称号由证据自动授予（例：「RAG 检索 · 已验证」），依据是被采纳的回答、跨部门安装、办过的活动，每条都能点开核实； 滚动 90 天，证据不再满足就静默摘掉 ，不通知、不公示。荣誉称号（季度社区之星、专栏作者）由评选授予，永久保留。
精华与专栏 曝光位	每周精选挂标 + 首页位置 + 进当周速递； 新作者有保底轮播位 。精华作品自动进知识库，可被检索和引用。
AI 额度 路线 C 的产品化	按等级和成就 自动加发 ，无需审批。技术上要新建一张持久化的用量账本 —— 现在的限流是 进程内存计数，重启就清零、多进程不共享 ，做不了按人配额；这是这项里真正的工作量。规则页全公开，可自查明细。

所有计分共用一套防刷内核：**每类行为按日封顶、同类重复按 $1/\sqrt{N}$ 递减、跨部门受益加权、只认被消费的贡献。**

【新功能 10】专家专栏：把路线 B 装进社区，而不是另起一个站

有稿就发，没稿不塌 —— 这是它和"独立立项"最大的区别

功能：用户看到什么

路线 B 的落点

- 社区里的一个「专栏」标签，加首页一个固定位置。约稿 + 自荐，管理员择优。
- 专栏文章自动进知识库：有 AI 逐章导读、可被检索、可被 AI 引用，读者能直接对着文章提问。
- 作者认领与署名。内部技术报告的作者用工号自动匹配认领，页面挂"作者已认证"，被讨论时通知作者本人 —— 作者在场是学术社区最强的参与诱因。
- 作者的回报：荣誉称号、被引用次数、在名片上的专栏作品集。对工程师，这些比稿费更好使。

技术：怎么做的

零新表

- 不新建内容体系。专栏就是打了标的帖子 / 文档，复用现有的可见性三级、评论线程、精华挂标；专栏页 = 一次带过滤条件的查询。
- 入库复用知识库的抽取管线：贴链接或传 PDF 就能章节化、生成导读、切分成可检索的片段，这条链路已经在跑。
- 作者认领只认工号严格匹配，绝不按用户名匹配 —— 开放注册环境下，按用户名匹配等于让人冒领别人的作品，这个安全口径在员工名单同步里已经定死。
- 被引用计数与第 12 项的 agent 引用共用一张计数表，专栏文章被 AI 引用同样计入作者。

和 2012 / 稼先不比谁更权威，比 谁离你今天的活更近：内网模型怎么用、我们自己的坑怎么填。

【新功能 11】语义搜索地基：一次投入，五个地方受益

这是全部规划里唯一一个有硬外部依赖的项目

功能：用户看到什么

基础设施

- **搜得到。**换个说法也能搜到，不用猜对方当初用了哪个词。
- **发帖前先给答案。**标题写完停 800 毫秒，相似的已解决主题直接亮在编辑框旁边 —— 拦住重复提问，比拦在事后有用。
- **跨类型相关推荐。**看完一篇文档，下面是相关讨论、相关 Skill、相关活动，把"看 → 用"的链路打通。
- **找人也更准：**专家匹配、话题聚合、缺口聚类都靠它。

技术：怎么做的

唯一的外部依赖

- **全站内容统一向量化**，存进数据库的向量扩展；写入时更新，存量夜间限速回填。
- **可见性快照跟着向量走：**内容转为私有时同步清除向量，检索结果再按查询者权限过滤一次 —— 双保险，因为搜索是最容易漏权限的地方。
- **只提示不拦截。**查重结果只是编辑框旁边的提示，永不阻止发布；阈值先灰度再放开。
- **硬依赖：**内网 vLLM 上加载一个 **embedding 模型**（bge-m3 / Qwen3-Embedding 级别，约 0.5—1B，单卡 8—16G 显存就够）。模型不大，但排期不由我们决定 —— **这是需要提前排队的一件事。**
- **排不上也有降级路径：**先用大模型分组做缺口聚类、用现有关键词搜索兜底，功能打折但不阻塞。

不先做它，问答 AI、话题、专家匹配都只能跑降级版 —— 所以它要早排队，不一定要早上线。

【新功能 12】加分项：MCP 知识出口 + 双周挑战赛

一个是别家做不了的技术故事，一个是激活潜水用户最省钱的一招

MCP 知识出口

让知识被 agent 直接消费

- 同事在自己的 Coding Agent 里一行配置接入，就能检索社区里已验证的问答、知识库文档和 Skill 说明，答案带引用。
- 作者能看到"我的回答被 agent 引用了 37 次" —— 这是 2025 年之后才出现的一种贡献度量，最对工程师胃口。
- 技术上：用 Next.js 路由实现流式 MCP 服务端；鉴权直接复用现有的命令行令牌体系（只读权限 + 限流 + 可审计的调用日志），账号体系不用重建 —— 别家做这件事要从零建。
- 密级第一：只出口公开且已验证的内容，服务端强制按权限裁剪，宁可少出口。

双周挑战赛

把"只看不发"的人拉进来

- 两周一题：给指定场景写一个 Skill、提示词攻防、复现一篇论文、用内网模型解一个真实工单。
- 投票 + 评审双轨评优，获奖必须写一篇复盘并进知识库 —— 这条规则是 Kaggle 文化里最值钱的部分。
- 技术上几乎零新建：活动、投票、评论、精选合集包全是现成的能力，把它们串成一条赛制流程即可。
- 和激励打通：获奖进当周速递头条、作品收进精选合集、作者拿荣誉称号和 AI 额度。

这两项都排在最后，但 MCP 出口是对外讲得最响的一个故事 —— 社区从"给人读"变成"人机共读"。

【路线】 三批交付：和"让人可见 → 让发容易 → 让留有理由"一一对应

分批依据是依赖顺序，不是重要程度

批次	周期	内容	为什么排在这一批
第一批 让人可见 + 让发容易	3 – 5 周	关注与动态流 + @提及 · 个人名片与本周红人 · 统一话题 · 一键发日常 + AI 顺稿 · 新人首帖引导 · 一页速递与每周精选	六项全部 零外部依赖 ，可以立即开工。它们合起来才构成"发一条有回响"这个最小闭环 —— 缺任何一项，其余几项都打折。运营上同时启动：3–5 名种子活跃者 + 每周一次精选。
第二批 让留有理由	1 – 2 个月	问答终态（采纳）· AI 值班员 · 贡献激励（等级 / 成就 / 称号 / 精华）+ AI 额度挂钩 · 专家专栏 · 语义搜索地基	要等第一批攒下真实的行为数据，激励才有得算；AI 值班员和搜索都 依赖向量模型的算力排期 ，所以这条要今天就去排队。专栏在这一批开栏，稿源慢慢约。
第三批 加分项	后续季度	MCP 知识出口 · 双周挑战赛 · 悬赏榜（揭榜挂帅）· 贡献档案导出	都依赖前两批积累的已验证内容和运营节奏。MCP 出口是对外最响的技术故事，但空社区出口出去也没有价值。

第一批的验收（上线四周后回答三个问题）：有多少人发出了第一条内容、有多少条内容拿到了 3 个以上的互动、速递带来多少次回站点击。数据不行就调机制，不加功能。

不做事：这一轮不新增板块、不做私信、不做全站总分排行榜、不做等级降级。功能面已经够了，缺的是机制。

三批的顺序不能换：**人不可见的时候做激励，等于给空房间发奖品。**

【度量】北极星是"周活跃发帖人数"，不是访问量

访问量涨得快也掉得快；只有持续有人往里放东西，社区才立得住

指标	口径	1 个月	3 个月	6 个月
★ 周活跃发帖人数	本周发过动态 / 话题 / 回复 / 短视频任一项的人数	出基线	基线的 3 倍	基线的 6 倍
首次发帖转化率	登录过的人里，发过至少一条内容的比例	出基线	≥ 25%	≥ 40%
帖均互动人数	一条内容平均有多少个不同的人反应或评论（衡量"发了有没有人理"）	出基线	≥ 3	≥ 5
跨部门互动比例	互动双方不在同一部门的比例（衡量"有没有真的认识的组的人"）	出基线	≥ 30%	≥ 40%
零回复率	发出去 24 小时内没有任何人回应的内容占比（越低越好，直接对应"第一次就是最后一次"）	出基线	≤ 20%	≤ 10%
速递回站率	由速递进入站内的点击人数 / 发送人数（内网邮件没有打开追踪，只统计点击）	—	≥ 20%	≥ 25%

基线怎么来：这几个数用管理后台现有数据回溯最近 30 天就能算出，会前出数、写进纪要冻结；种子活跃者带来的量单独标注，不计入分子分母，否则目标没有意义。

数不出来但更要紧的：新人第一天就知道"这个方向该找谁"；同事之间能叫得出名字；有人因为社区里被看见而多写了一篇 —— 这三件事发生了，数字才会跟着来。

度量口径先定，功能再上。没有基线，三个月后没法判断这批投入值不值。

【诉求】 需要拍板的四件事

为什么是现在：平台的建设成本已经投进去了，机制层不补，这部分投入会随活跃度下滑一起沉掉

#	请示事项	用来做什么	拿不到会怎样
1	AI 额度池的授权 额度归谁、一个季度多少	把贡献和 AI 资源挂钩（路线 C）：被采纳、被安装、被感谢的人自动加发额度，全自动无需审批	激励只剩荣誉一层。这是我们手上 最独特也最有效的一张牌 ，不用等于浪费
2	内网加载一个向量模型 bge-m3 / Qwen3-Embedding 级别，约 0.5-1B，单卡 8-16G 显存	语义搜索地基：搜得到、发帖前查重先给答案、跨类型推荐、AI 值班员的全站检索	第二批一半功能走降级实现； 排队时间不由我们决定，建议本周就去排
3	运营：0.3-0.5 人力 + 3-5 名 种子活跃者	每周一次精选与红人榜（10 分钟）· 每两周组织一个话题 · 新人首帖 24 小时内有人回 · 种子活跃者首月每人每周发 2 条	机制再好也需要人先把水搅热。 这是最大的单点风险 ，也是最便宜的一项
4	一到两个试点部门，主管带头发	首月在两个部门内跑通"发一条有人理"，把示范做出来再向外扩	社区型产品的成败在前 30 天，冷启动期没有示范，后面再补要贵得多

总投入：开发由现有 1 人承担 —— 第一批 3-5 周、第二批 6-8 周，**不新增编制**；代价是同期其他任务顺延。除额度池外没有新增预算。

预计收益：社区活起来之后最直接的三项 —— 重复造的 Skill 变成一次上传多次安装、长文档一人读完多人可问、找人从"群里问一圈"变成"搜一下"。按首年 60 个跨部门 Skill、200 条沉淀问答估，量级在千人时。

止损线：第一批上线四周后，若发过第一条内容的人数 < __、零回复率仍 > __%，就停在第一批不再追加投入，已上线功能维持现状。

四件里最急的是第 2 条：**算力排期的等待时间不由我们决定，越早排队，第二批越不容易被卡住。**

Thank you.

六个板块已经建成并在内网运行；接下来这一轮不加板块，只做一件事 —— 让每个人都愿意在这儿发第一条。

现场提问，或者会后找我，我把第一批的详细实施方案发给你。

